

PyTorch Comprehensive Cheatsheet

1. Core Tensor Operations

```
python
import torch          # Main PyTorch library
from torch import Tensor  # Base tensor class

# Tensor creation:
torch.zeros(), torch.ones(), torch.rand(), torch.randn(),
torch.arange(), torch.linspace(), torch.tensor(), torch.empty()

# Tensor operations:
torch.cat(), torch.stack(), torch.split(), torch.chunk(),
torch.reshape(), torch.transpose(), torch.permute(),
torch.matmul(), torch.mm(), torch.bmm(), torch.einsum()

# Tensor indexing:
torch.masked_select(), torch.index_select(),
torch.gather(), torch.scatter()
```

```
# Reduction ops:
torch.sum(), torch.mean(), torch.min(), torch.max(),
torch.argmax(), torch.argmin(), torch.std(), torch.var()
```

```
# Math ops:
torch.exp(), torch.log(), torch.sqrt(), torch.sin(),
torch.cos(), torch.abs(), torch.clamp(), torch.sigmoid(),
torch.softmax(), torch.log_softmax()
```

2. Autograd (Automatic Differentiation)

```
python
from torch.autograd import Variable  # Deprecated (use tensors directly)
from torch.autograd import grad      # Compute gradients manually
x = torch.tensor(..., requires_grad=True)  # Enable gradient tracking
x.backward()                          # Compute gradients via backprop
torch.autograd.gradcheck()            # Check gradient computation
torch.autograd.gradgradcheck()       # Check higher-order gradients
```

3. Neural Network Module (nn)

```
python
import torch.nn as nn  # Neural network layers, loss functions, etc.
```

```
# Common Layers:
nn.Linear(), nn.Conv1d(), nn.Conv2d(), nn.Conv3d(),
nn.LSTM(), nn.GRU(), nn.RNN(), nn.Embedding(),
nn.BatchNorm1d(), nn.BatchNorm2d(), nn.BatchNorm3d(),
nn.Dropout(), nn.Dropout2d(), nn.Dropout3d(),
nn.ReLU(), nn.Sigmoid(), nn.Tanh(), nn.LeakyReLU(),
nn.Softmax(), nn.LogSoftmax(), nn.Flatten()
```

Copy

Copy

Copy

Loss Functions:

```
nn.MSELoss(), nn.CrossEntropyLoss(), nn.NLLLoss(),  
nn.BCELoss(), nn.BCEWithLogitsLoss(), nn.KLDivLoss(),  
nn.HingeEmbeddingLoss(), nn.CosineEmbeddingLoss(),  
nn.TripletMarginLoss(), nn.PoissonNLLLoss()
```

Utilities:

```
nn.Module                # Base class for custom models  
nn.Sequential()          # Container for stacking layers  
nn.Parameter()           # Trainable parameters in custom layers  
nn.init                  # Weight initialization  
nn.init.xavier_uniform_(), nn.init.kaiming_normal_(),  
nn.init.zeros_(), nn.init.ones_()
```

4. Optimizers

```
python  
import torch.optim as optim    # Optimization algorithms
```

Copy

Common Optimizers:

```
optim.SGD(), optim.Adam(), optim.RMSprop(),  
optim.Adagrad(), optim.Adadelta(), optim.AdamW(),  
optim.LBFGS(), optim.ASGD()
```

Learning Rate Schedulers:

```
optim.lr_scheduler.StepLR(), optim.lr_scheduler.CosineAnnealingLR(),  
optim.lr_scheduler.ReduceLROnPlateau(), optim.lr_scheduler.CyclicLR()
```

5. Data Loading & Processing

```
python  
from torch.utils.data import Dataset, DataLoader # Custom datasets + batching  
from torch.utils.data import random_split, Subset # Dataset utilities  
from torchvision import transforms              # Image preprocessing
```

Copy

Example:

```
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize(mean=[0.5], std=[0.5]),  
    transforms.RandomHorizontalFlip(),  
    transforms.RandomRotation(30)  
])
```

Dataset utilities:

```
TensorDataset, ConcatDataset, ChainDataset
```

6. Distributed Training

```
python
```

Copy

```
import torch.distributed as dist          # Distributed communication
from torch.nn.parallel import DistributedDataParallel as DDP # Multi-GPU training
from torch.utils.data.distributed import DistributedSampler # Distributed data sampling
torch.distributed.init_process_group()     # Initialize distributed backend
```

7. TorchScript (Model Deployment)

```
python
from torch.jit import script, trace # Convert models to TorchScript
model_scripted = torch.jit.script(model) # Scripted model for deployment
model_traced = torch.jit.trace(model, example_input) # Traced model for deployment
torch.jit.save(model_scripted, "model.pt") # Save TorchScript model
```

8. ONNX Export

```
python
import torch.onnx          # Export models to ONNX format
torch.onnx.export(model, ...) # Convert PyTorch model to ONNX
```

9. CUDA & Device Management

```
python
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
x = x.to(device)          # Move tensors to GPU/CPU
torch.cuda.empty_cache()   # Clear GPU memory
torch.cuda.memory_allocated() # Check GPU memory usage
torch.cuda.memory_reserved() # Check GPU memory reserved
```

10. Debugging & Profiling

```
python
from torch.profiler import profile, record_function # Performance profiling
torch.autograd.profiler.profile()                  # Autograd profiling
torch.autograd.set_detect_anomaly(True)             # Detect NaN/inf in gradients
```

11. Vision & Text Libraries (External)

```
python
# Install separately: pip install torchvision torchtext
import torchvision          # For CV (datasets, models, transforms)
import torchtext            # For NLP (text preprocessing, datasets)
```

TorchVision:

```
from torchvision import datasets, models, transforms
from torchvision.models import resnet18, vgg16, mobilenet_v2
```

TorchText:

```
from torchtext.data import Field, BucketIterator, TabularDataset
from torchtext.vocab import GloVe, FastText
```

12. Utilities

```
python
from torch.utils.data import TensorDataset # Wrap tensors into a dataset
from torch.utils.data import random_split # Split datasets
from torch.utils.tensorboard import SummaryWriter # Logging for TensorBoard
from torch.utils.checkpoint import checkpoint # Gradient checkpointing
```

13. Specialized Libraries

```
python
# PyTorch Geometric (Graph Neural Networks):
# Install: pip install torch-geometric
import torch_geometric
from torch_geometric.data import Data, Dataset
from torch_geometric.nn import GCNConv, GATConv
```

Copy

```
# PyTorch Lightning (High-level training framework):
# Install: pip install pytorch-lightning
import pytorch_lightning as pl
from pytorch_lightning import Trainer
```

```
# FastAI (High-level API):
# Install: pip install fastai
from fastai.vision.all import *
from fastai.text.all import *
```

14. Miscellaneous

```
python
# Random number generation:
torch.manual_seed(42)
torch.cuda.manual_seed_all(42)
```

Copy

```
# Save and load models:
torch.save(model.state_dict(), "model.pth")
model.load_state_dict(torch.load("model.pth"))
```

```
# Serialization:
torch.save(), torch.load(), torch.jit.save(), torch.jit.load()
```

Key Tips

- . **Device Agnostic Code:** Always use `.to(device)` for tensors/models.
- . **Use `nn.Module`** : Structure models by subclassing `nn.Module` .
- . **Batch First:** Most layers expect input shape `(batch_size, ...)` .
- . **Seed for Reproducibility:**

```
python
torch.manual_seed(42)
torch.cuda.manual_seed_all(42)
```

Copy